

Architettura degli Elaboratori

Funzioni, activation record e indirizzi



SAPIENZA
UNIVERSITÀ DI ROMA

Alessandro Checco

checco@di.uniroma1.it

Special thanks and credits:

Andrea Sterbini, Iacopo Masi, Claudio di
Ciccio

[S&PdC]

Capitolo 2

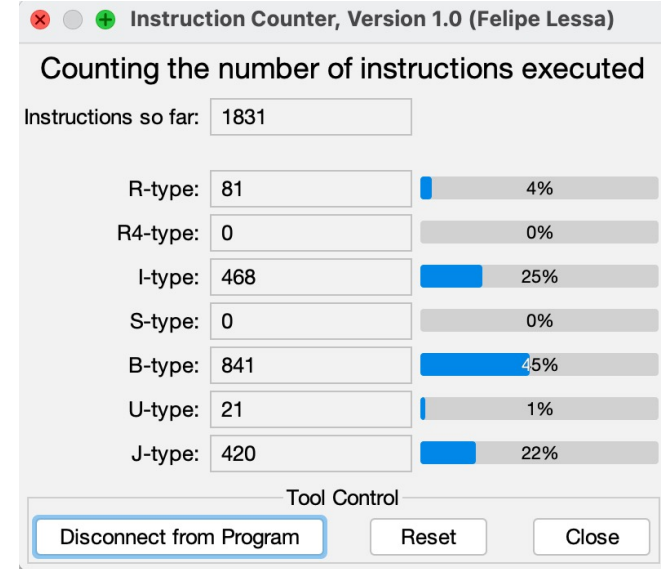
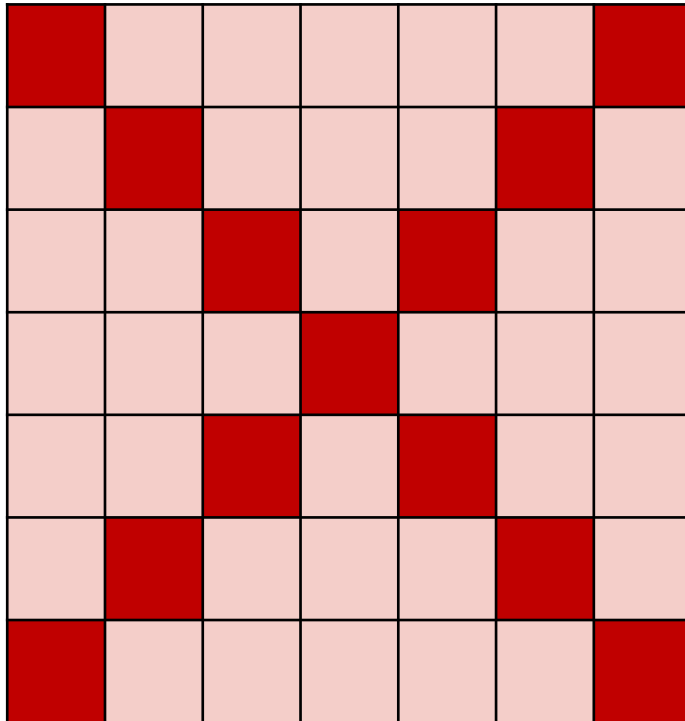


Esercizio



SAPIENZA
UNIVERSITÀ DI ROMA

Somma della diagonale (v. più efficiente)

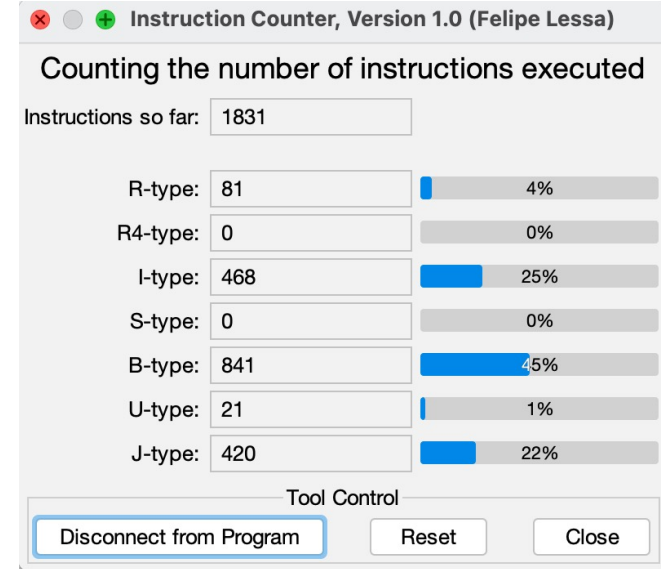
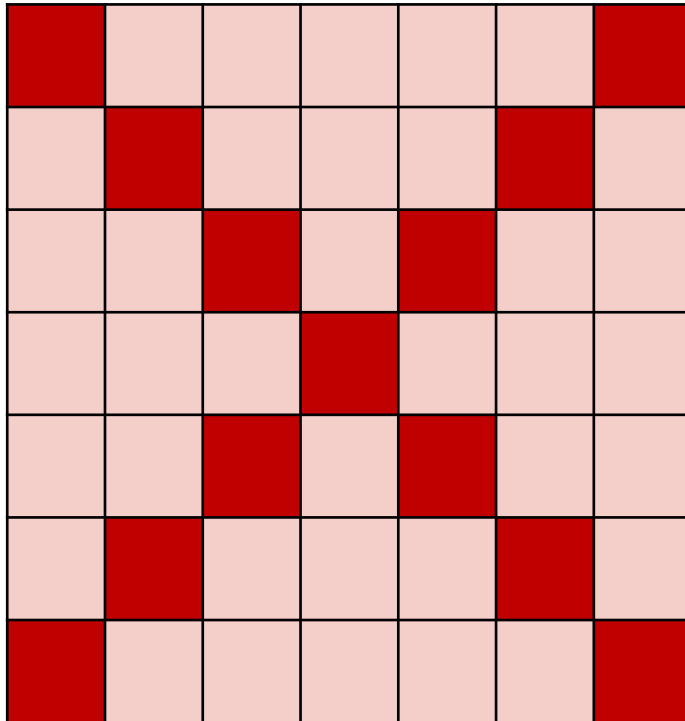


Esercizio 1:

Scrivere un programma in assembly che, data una matrice bidimensionale quadrata MATRIX di word in memoria, di lato N, restituisca in s0 il risultato della somma dei valori sulle diagonali principale e secondaria.



Somma della diagonale (v. più efficiente) / reprise



Esercizio 2:

Aggiungere istruzioni di stampa e di uscita al programma di cui all'esercizio 1.



Circa gli spazi di indirizzamento e i salti



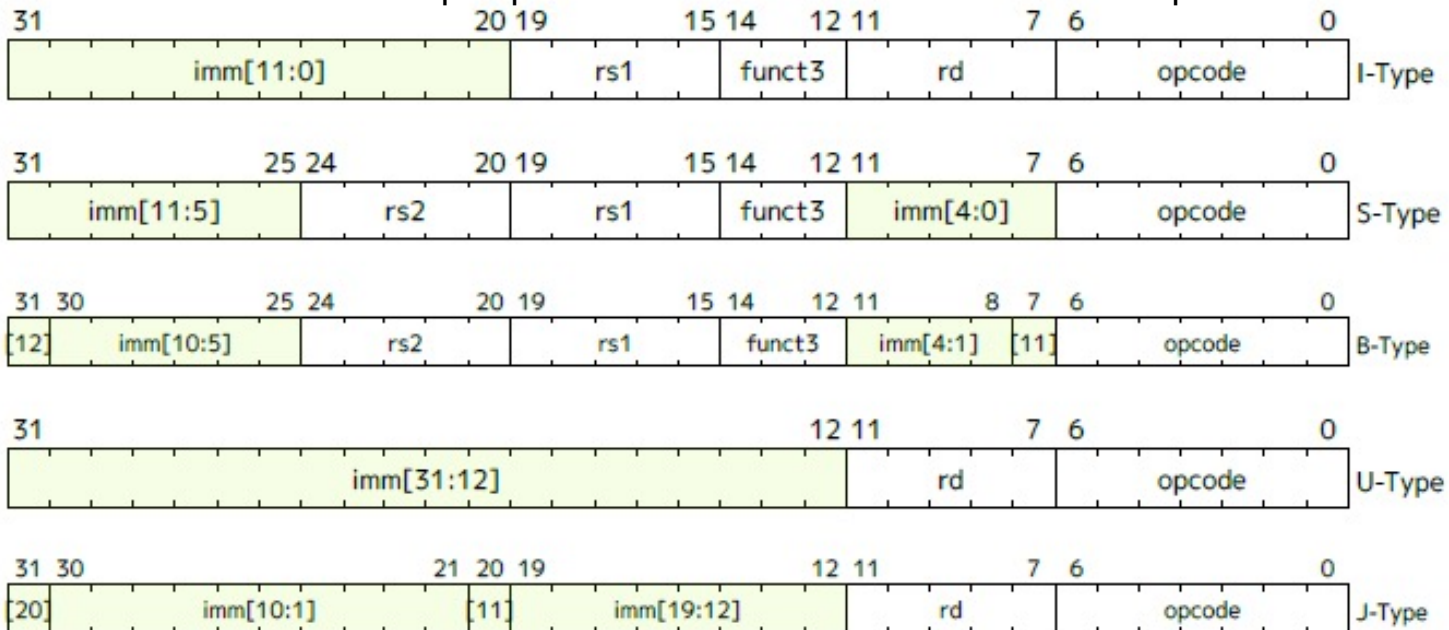
SAPIENZA
UNIVERSITÀ DI ROMA

Indirizzamento RISC-V immediato a 32bit

Anche se mantenere tutte le istruzioni a 32-bit semplifica l'hardware, ci sono dei casi in cui è utile passare una costante o un indirizzo a 32bit in un'istruzione.

Il RARS può indirizzare 2^{32} byte.

Nessuna istruzione offre 32 bit per puntare alle locazioni in memoria disponibili.



Idea: «spezzare» il caricamento in più istruzioni.

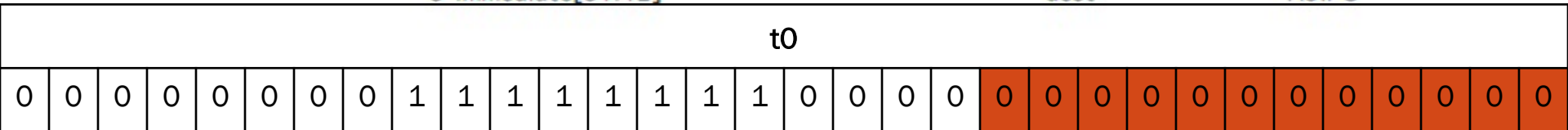
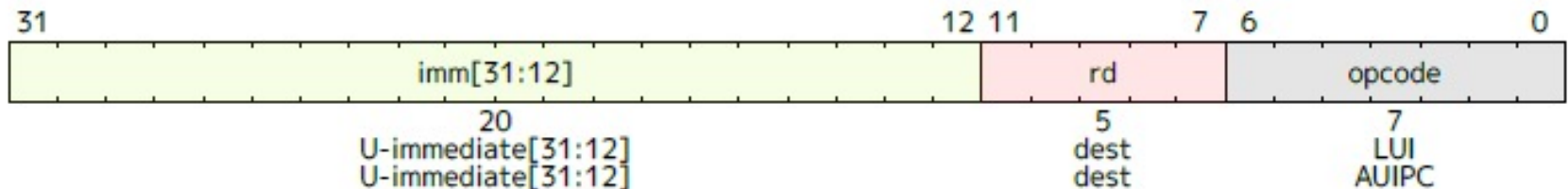
Load upper immediate (lui) + addi immediate (addi)

Qual è il codice assembly RISC-V usato per caricare la seguente costante a 32 bit nel registro s0?

0000 0000 1111 1111 0000 1001 0000 0000

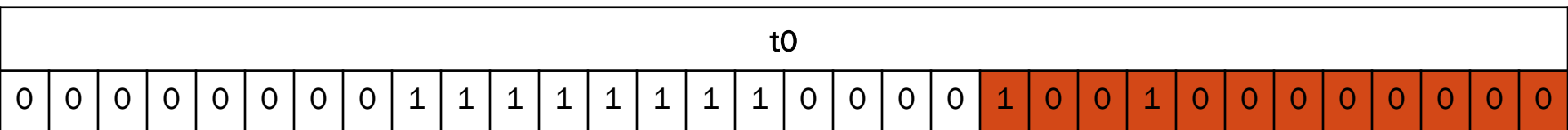
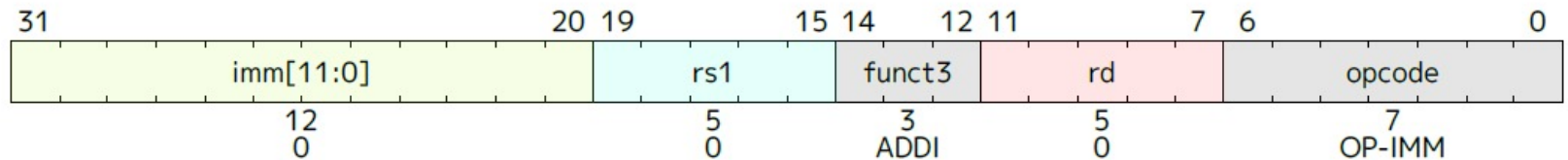
1

lui t0, 0x00FF0



2

addi t0, 0x900



Chiarimenti su indirizzamento nei salti

Address	Code	Basic	Source
4194304	0x00000013	addi x0,x0,0	3: main: li x0,0
4194308	0x02000663	beq x0,x0,44	4: beqz x0, hello
4194312	0x00030313	addi x6,x6,0	5: addi t1,t1,0
4194316	0x00030313	addi x6,x6,0	6: addi t1,t1,0
4194320	0x00030313	addi x6,x6,0	7: addi t1,t1,0
4194324	0x00030313	addi x6,x6,0	8: addi t1,t1,0
4194328	0x00030313	addi x6,x6,0	9: addi t1,t1,0
4194332	0x00030313	addi x6,x6,0	10: addi t1,t1,0
4194336	0x00030313	addi x6,x6,0	11: addi t1,t1,0
4194340	0x00030313	addi x6,x6,0	12: addi t1,t1,0
4194344	0x00030313	addi x6,x6,0	13: addi t1,t1,0
4194348	0x00030313	addi x6,x6,0	14: addi t1,t1,0
4194352	0x0040006f	jal x0,4	15: hello: j ok
4194356	0x00000317	auipc x6,0	16: ok: la t1,main
4194360	0xfcc30313	addi x6,x6,-52	
4194364	0x00030067	jalr x0,x6,0	17: jr t1

4194308 + 44 = 4194352 (relativo)

jump to PC+4 (relativo)

$x6 = PC + 0 = 4194356 + 0$

$x6 = x6 - 52 = 419304$

(usato il PC per costruire un indirizzo di salto

da mettere del registro t1)

Salti di tipo condizionato e jal sono relativi al PC

Salti jalr (e jr) sono assoluti, ma possono essere costruiti dall'assemblatore a partire del PC

Funzioni, invocazioni e activation record



SAPIENZA
UNIVERSITÀ DI ROMA

Funzioni: ingredienti – Istruzioni necessarie

Per chiamare la funzione/procedura

jal *etichetta* Jump And Link

registra nel registro **ra** la posizione dell'istruzione successiva ($ra \leftarrow PC+4$)

cambia il PC per iniziare l'esecuzione del corpo della funzione ($PC \leftarrow PC+etichetta$)

L'etichetta indicizza alla mezza parola

Per tornare e continuare l'esecuzione del programma chiamante

jr **ra** Jump to Register

salta all'indirizzo contenuto nel registro indicato ($PC \leftarrow ra$)

Come passare valori ALLA funzione (caso semplice)

a0-a7 8 registri per passare fino a 8 valori a 32 bit

e restituirli DALLA funzione

a0, a1 2 registri per restituire fino a 2 valori a 32 bit

In quali problemi incorriamo usando solo **jal** e **jr ra** se quando si vogliono implementare funzionalità complesse?

Istruzioni necessarie

Per chiamare la funzione/procedura

jal rd, etichetta Jump And Link

ricorda nel registro **rd** la posizione dell'istruzione successiva ($rd \leftarrow PC+4$)

cambia il PC per iniziare l'esecuzione del corpo della funzione ($PC \leftarrow PC + \text{etichetta}$)

j etichetta $\rightarrow$ **jal x0, etichetta**

jal etichetta $\rightarrow$ **jal ra, etichetta**

Per tornare e continuare l'esecuzione del programma chiamante

jalr rd, rs1, +100 Jump to Register

ricorda nel registro **rd** la posizione dell'istruzione successiva ($rd \leftarrow PC+4$)

salta all'indirizzo contenuto nel registro indicato con spiazzamento ($PC \leftarrow rs1 + 100$)

jr rs1, -100 $\rightarrow$ **jalr x0, rs1, -100**

Come passare valori ALLA funzione (caso semplice)

a0, a1, ... , a7 8 registri per passare e restituire 8 valori a 32 bit

e restituirli DALLA funzione

a0, a1 2 registri per restituire fino a 2 valori a 32 bit

nota (convenzioni):

t0, t1, ... possono cambiare tra una chiamata e l'altra (temporary)

s0, s1, ... non cambiano tra una chiamata e l'altra (saved)

Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante
- nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di **ra**
- **a0, a1, ..., a7** possiamo solo passare 8 argomenti

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- salvo stato prima di chiamata 1
 - salvo stato prima di chiamata 2
 - ...
 - ripristino stato prima di chiamata 2
 - ripristino stato prima di chiamata 1

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro **sp** (Stack Pointer)

Esempio chiamata funzioni annidate

- main chiama foo che chiama bar
- foo ha bisogno di 3 registri s0, s1, s2
- bar ha bisogno di 2 registri s0, s1
- return address? (se ho già nidificato lo perdo)

Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante
- nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di **ra**
- **a0, a1, ..., a7** possiamo solo passare 8 argomenti

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- vengono salvate le prime
 - ne vengono salvate altre
 - ...
 - vengono prima ripristinate le seconde
- e poi vengono ripristinate le prime

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro **sp** (Stack Pointer)

sp

980

Address	MEM
1000	xxxx
996	yyyy
992	zzzz
988	tttt
984	ssss
980	rrrr
976	
972	
968	
964	
960	
956	
952	
948	
944	
940	
936	
932	
928	
924	

Uso dello stack (implementare push e pop)

Lo stack si trova nella parte «alta» della memoria e cresce **verso il basso**

Supponiamo di voler salvare e ripristinare il registro **ra**

Come salvare un elemento (push):

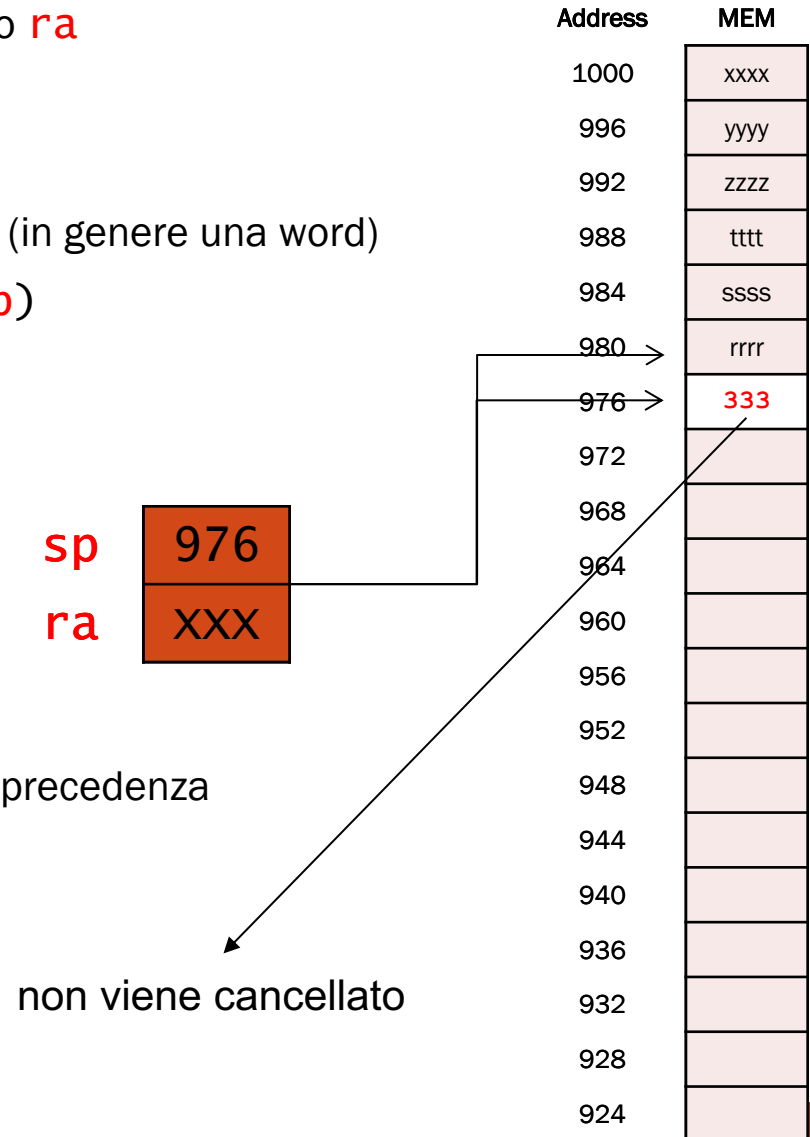
- si **decrementa** lo **sp** della dim. dell'elemento (in genere una word)
- si memorizza l'elemento nella posizione **0(sp)**

```
addi    sp, sp, -4
sw      ra, 0(sp)
```

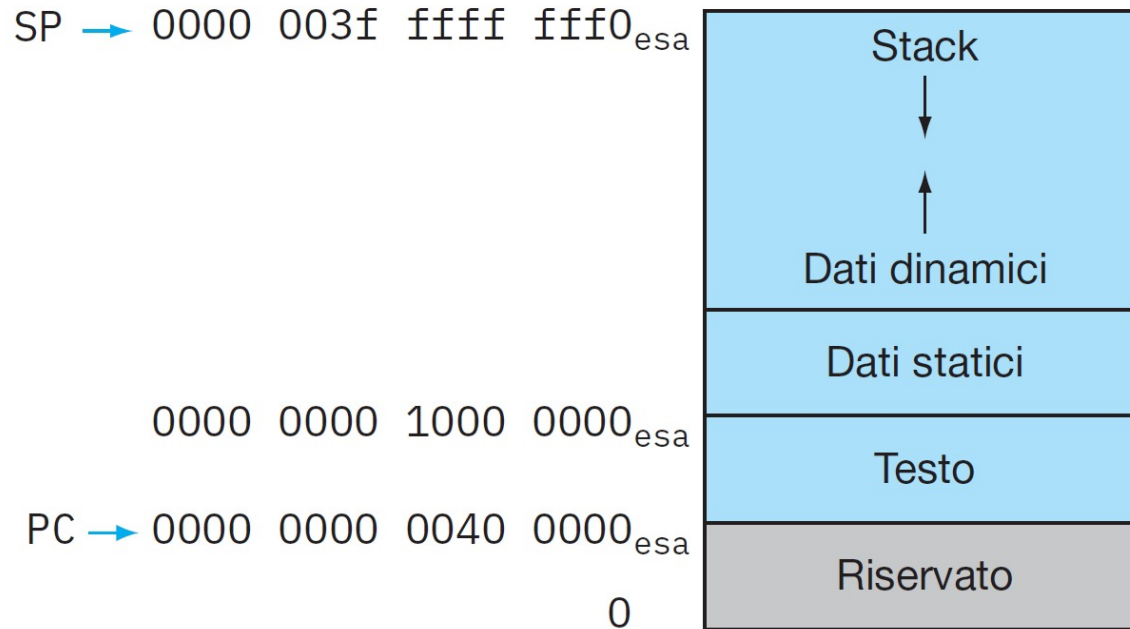
Come recuperare un elemento (pop)

- si legge l'elemento dalla posizione **0(sp)**
- si **incrementa** lo **sp** della quantità allocata in precedenza

```
lw      ra, 0(sp)
addi    sp, sp, 4
```



La memoria



Uso dello stack in una funzione

funzione:

All'inizio della funzione:

- **allocare** su stack abbastanza word da contenere i registri da preservare

- **salvare** su stack i registri, ad offset multipli di 4 rispetto a **sp**

addi **sp**, **sp**, -12

sw **ra**, 8(**sp**)

sw **s0**, 4(**sp**)

sw **s1**, 0(**sp**)

corpo della funzione

posso chiamare jal

tranquillamente perché

salvo ra

NOTA: conviene allocare tutto lo spazio assieme per avere **offset che restano costanti** durante tutta l'esecuzione della funzione

All'uscita della funzione:

- **ripristinare** da stack i registri salvati, agli stessi offset usati precedentemente

- **disallocare** da stack lo stesso spazio allocato in precedenza

- **tornare** alla funzione chiamante

lw **s1**, 0(**sp**)

lw **s0**, 4(**sp**)

lw **ra**, 8(**sp**)

addi **sp**, **sp**, 12

jr **ra**

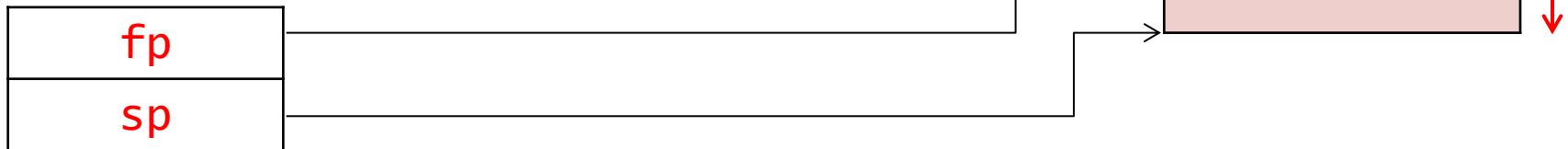
Record di attivazione (stack frame / activation record)

Lo stack è usata anche per:

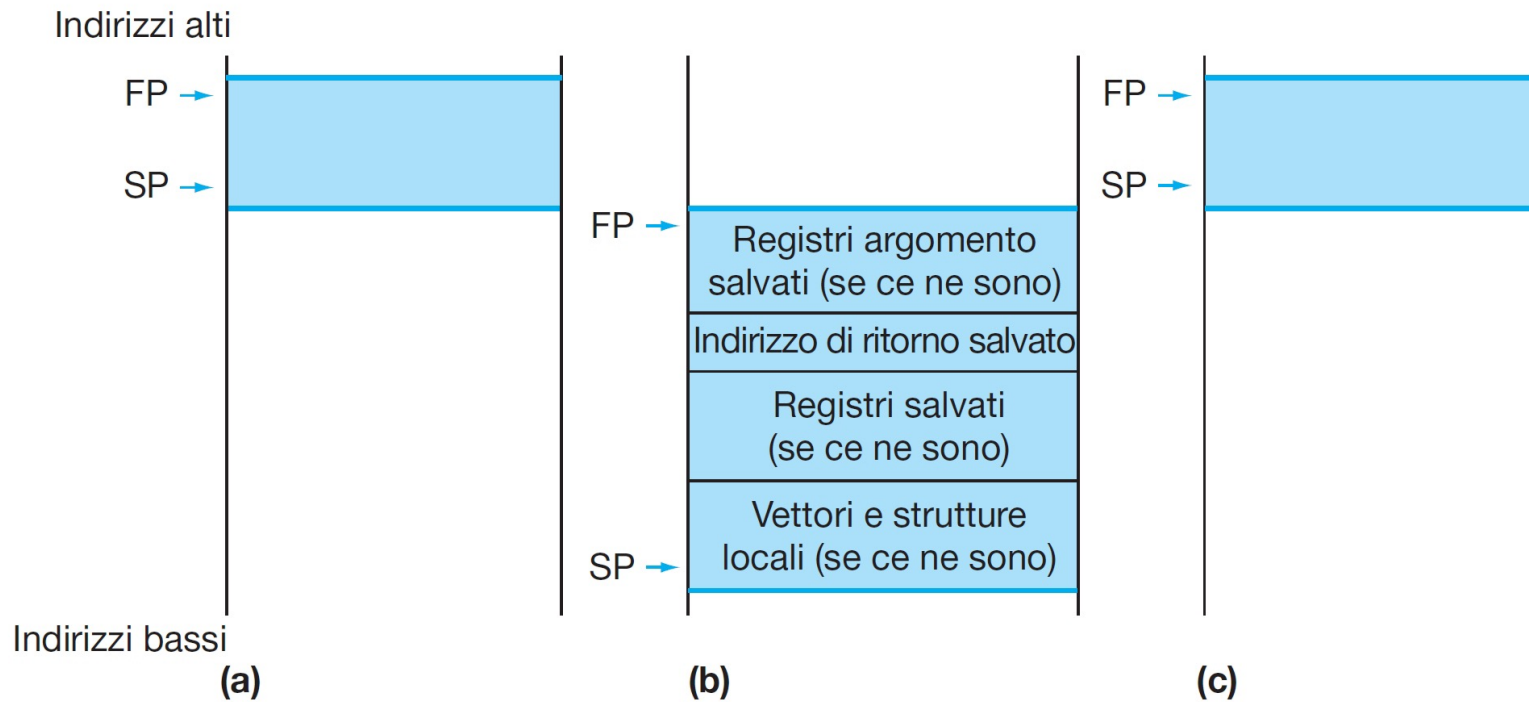
- Comunicare **ulteriori argomenti** oltre $a_0, \dots, a_7$
- Comunicare **ulteriori risultati** oltre a_0, a_1
- Allocare **variabili locali** alla procedura

Questo blocco prende il nome di **stack frame** o **activation record**)

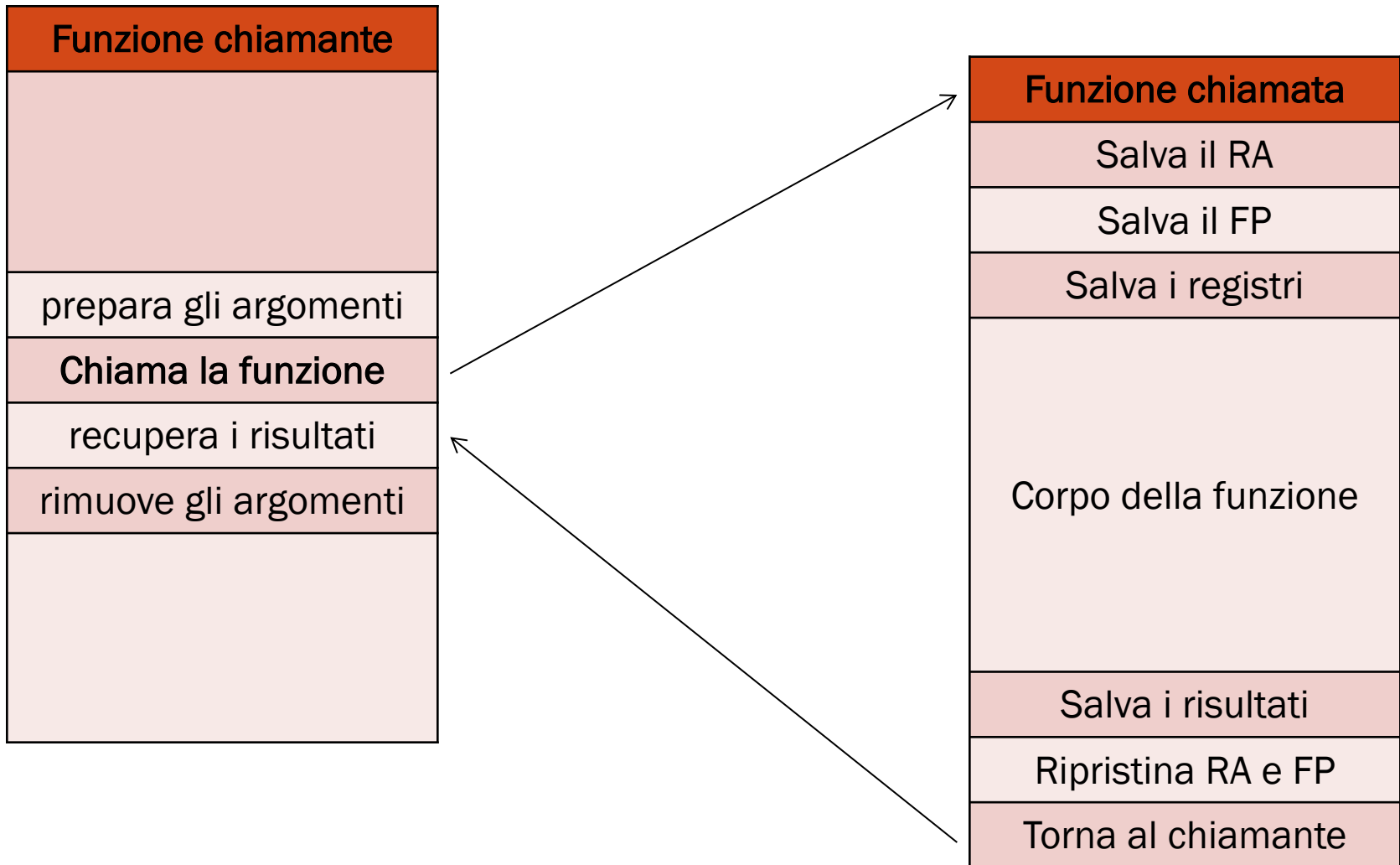
- Allocated su stack *prima* della chiamata della funzione e rilasciato subito dopo
- **sp (stack pointer)** si usa per puntare alla fine del record di attivazione
 - Varia allocando dati dinamicamente
- **fp (frame pointer)** si usa per puntare all'inizio del record di attivazione (ridondante ma comodo, non molto usato)
 - Resta fisso durante l'esecuzione della funzione



Frame pointer e stack pointer



Schema di una chiamata



Esempio: calcolare f in RISC-V assembly

avgOfSquareAbsSub


$$f(x, y, w, z) = \frac{(|x| - |y|)^2 + (|w| - |z|)^2}{2}$$

squareAbsSub


$$g(x, y) = (|x| - |y|)^2$$

$$f(x, y, w, z) = \frac{g(x, y) + g(w, z)}{2}$$

Esempio: calcolare f in MIPS assembly

```

22 # function avgOfSquareAbsSub(a0: int, a1: int, a2: int, a3: int): (a0: int)
23 # returns ( ( |a0| - |a1| ) ^ 2 + ( |a2| - |a3| ) ^ 2 ) / 2
24 avgOfSquareAbsSub:
25     addi sp, sp, -28      # Move the stack pointer down
26     sw ra, 0(sp)         # First: save the return address
27     sw fp, 4(sp)         # Save fp
28     sw a0, 8(sp)         # Save a0
29     sw a1, 12(sp)        # Save a1
30     sw a2, 16(sp)        # Save a2
31     sw a3, 20(sp)        # Save a3
32     sw s1, 24(sp)        # Save s1
33
34     # First call to squareAbsSub with a0, a1 (already set)
35     jal squareAbsSub
36     mv s1, a0            # Save the temporary result in s1
37
38     # Second call with a2, a3
39     lw a0, 16(sp)        # Load a2 into a0
40     lw a1, 20(sp)        # Load a3 into a1
41
42     jal squareAbsSub
43     add a0, s1, a0        # Sum the temporary result
44     srai a0, a0, 1       # a0 = a0 / 2 (arithmetic shift right)
45
46     # Restore saved registers
47     lw s1, 24(sp)
48     lw fp, 4(sp)        # Recover fp
49     lw ra, 0(sp)        # Recover return address
50     addi sp, sp, 28      # Roll the stack pointer up
51     ret

```

perché salvo ra? *ers that we u* andrei in loop a 43 (jal sovrascrive ra)

non salvo ra?

foglia (non ho jal)!

```

53 # function squareAbsSub(a0: int, a1: int): (a0: int)
54 # returns ( |a0| - |a1| ) ^ 2
55 squareAbsSub:
56     stack (lo
57     scrivo dopo)
58     # Calculate |a0|
59     bge a0, zero, pos_a0 # If a0 >= 0, skip negation
60     neg a0, a0           # a0 = -a0 (absolute value)
61 pos_a0:
62     # Calculate |a1|
63     bge a1, zero, pos_a1 # If a1 >= 0, skip negation
64     neg a1, a1           # a1 = -a1 (absolute value)
65 pos_a1:
66     sub a0, a0, a1       # a0 = |a0| - |a1|
67     mul a0, a0, a0       # a0 = (|a0| - |a1|)^2
68
69     # Restore parameters (but don't overwrite return value in a0)
70     stack (dopo)
71     ret

```

Esempio: calcolare f in assembly RISC-V

```
1  .globl main
2  .data
3  A:  .word 5
4  B:  .word 7
5  C:  .word 12
6  D:  .word -8
7  .text
8  main:
9      lw a0, A
10     lw a1, B
11     lw a2, C
12     lw a3, D
13
14     jal avgOfSquareAbsSub # Call avgOfSquareAbsSub
15
16     li a7, 1
17     ecall
18
19     li a7, 10
20     ecall

22 # function avgOfSquareAbsSub(a0: int, a1: int, a2: int, a3: int): (a0: int)
23 # returns ( ( |a0| - |a1| ) ^ 2 + ( |a2| - |a3| ) ^ 2 ) / 2
24 avgOfSquareAbsSub:
25     addi sp, sp, -28 # Move the stack pointer down
26     sw ra, 0(sp)    # First: save the return address
27     sw fp, 4(sp)    # Save fp
28     sw a0, 8(sp)    # Save the parameters that we use locally
29     sw a1, 12(sp)   # (a0-a7 might be changed by nested call so I save them)
30     sw a2, 16(sp)   # but no need to restore them at the end
31     sw a3, 20(sp)
32     sw s1, 24(sp)   # supposed to be preserved after the call
33
34     # First call to squareAbsSub with a0, a1 (already set)
35     jal squareAbsSub
36     mv s1, a0        # Save the temporary result in s1
37
38     # Second call with a2, a3
39     lw a0, 16(sp)    # Load a2 into a0
40     lw a1, 20(sp)    # Load a3 into a1
41
42     jal squareAbsSub
43     add a0, s1, a0    # Sum the temporary result directly to a0
44     srai a0, a0, 1    # a0 = a0 / 2 (arithmetic right shift by 1)
45
46     # Restore saved registers
47     lw s1, 24(sp)
48     lw fp, 4(sp)    # Recover fp
49     lw ra, 0(sp)    # Recover return address
50     addi sp, sp, 28 # Roll the stack pointer back up
51     ret

53 # function squareAbsSub(a0: int, a1: int): (a0: int)
54 # returns ( |a0| - |a1| ) ^ 2
55 squareAbsSub:
56     addi sp, sp, -4 # Move the stack pointer down
57     sw fp, 0(sp)
58
59     # Calculate |a0|
60     bge a0, zero, pos_a0 # If a0 >= 0, skip negation
61     neg a0, a0           # a0 = -a0 (absolute value)
62 pos_a0:
63
64     # Calculate |a1|
65     bge a1, zero, pos_a1 # If a1 >= 0, skip negation
66     neg a1, a1           # a1 = -a1 (absolute value)
67 pos_a1:
68
69     sub a0, a0, a1       # a0 = |a0| - |a1|
70     mul a0, a0, a0       # a0 = (|a0| - |a1|)^2
71
72     # Restore parameters (but don't overwrite return value in a0)
73     lw fp, 0(sp)
74     addi sp, sp, 4
75     ret
```

in questo caso buona parte dello stack si poteva omettere,
ma è buona abitudine salvare tutto ciò che viene scritto
per rendere il codice riutilizzabile

Esempio in RARS

Registri: ricapitoliamo convenzioni

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller